You still have the Capybara integration test from Chapter 9, *Testing JavaScript: Integration Testing*, on page 181, and whatever you do here, that test should still work (or at least, should work pending minor changes based on changing DOM structure). So you can think of this process as a slightly different instance of outside-in testing. The Capybara test is still the outside, but instead of using RSpec to write server-side tests, you're using Jasmine to write client-side tests.

I should note that this is a very small feature, and I'm probably overbuilding it based on the current requirements, but it is handy to see a small example. In practice, the idea of using tests to isolate your JavaScript from the server and the DOM becomes more compelling the more logic is needed in the client. In practice, a larger example would likely use a framework like React or Vue that uses data binding to help isolate logic from the DOM.

You'll wind up with a Project and a Test class on the client that are not dissimilar to the ones you already have on the server. This duplication is a common problem in web applications, and there's not a clear solution. One solution is to move all the logic to the client and write a single-page web app backed by a simple server. This may be the right solution in some cases. In my experience, unless the client has a lot of interactivity, the client's single-page app winds up being more complicated than an app that has some logic on the server, some logic on the client, and some duplication. I realize that sounds counterintuitive.

Here's my first batch of Jasmine tests (note that I did write and get these to pass one at a time):

js_jasmine/01/spec/javascripts/project_spec.js

```
import {Project, Task} from "../../app/javascript/packs/projects.js"

describe("Projects", () => {
  let project

  beforeEach(() => {
    project = new Project()
    project.appendTask(new Task("Start Project", 1))
    project.appendTask(new Task("Middle Project", 2))
    project.appendTask(new Task("End Project", 3))
  })

  it("can identify the first element of a project", () => {
    expect(project.firstTask().name).toEqual("Start Project")
    expect(project.firstTask().isFirst()).toBeTruthy()
    expect(project.firstTask().isLast()).toBeFalsy()
    expect(project.firstTask().index).toEqual(0)
  })
```